קטע קוד A

```
class Expression {
  double value;
public:
  Expression(double v): value(v) {/* print(); cout << endl; */}
  virtual ~Expression() {/* cout << *this << endl; */}
  operator double () const { return value; }
  virtual void print(ostream& out = cout) const { out << value; }
  friend ostream& operator<<(ostream& out, const Expression& e) {
    e.print(out);
    out << " = " << e.value;
    return out;
  }
};

class Number: public Expression {
public:
    Number(double d) : Expression(d) {}
};

class Sum: public Expression {
  Expression *e1, *e2;
public:
  Sum(Expression* e1, Expression* e2):
    Expression(*e1 + *e2), e1(e1), e2(e2) {}
  ~Sum() override {
    delete e1; delete e2;
  }
  Sum(const Sum&) = delete;
  Sum& operator=(const Sum&) = delete;
  void print(ostream& out = cout) const override {
    out << '(';
    e1->print();
    out << " + ";
    e2->print();
    out << ')';
  }
};
```

להלן מספר main אפשריים:

A1 main

```
int main() {
  Expression* e = new Sum(
       new Number(3.5),
       new Sum(new Number(1), new Number(3))
    );
  cout << *e << endl;
  delete e;
}
```

A2 main

```
int main() {
  Number* n1 = new Number(3.5);
  Number n2 = *n1;
  cout << n2 << endl;
  delete n1;
}
```

A3 main

```
int main() {
  Expression* e1 = new Sum(new Number(1), new Number(2));
  Sum s2 = *(dynamic_cast<Sum*>(e1));
  cout << s2 << endl;
  delete e1;
}
```

A4 main

```
int main() {
  list<Expression*> expressions =
     {new Number(2.5), new Sum(new Number(1), new Number(2))};
  Expression* max = nullptr;
  for_each(expressions.begin(), expressions.end(),
      [&max](Expression* e){
          if(max == nullptr || *e > *max) {
             delete max;
             max = e;
          }
          else {
             delete e;
          }
     });
  cout << (double)*max;
  delete max;
}
```

קטע קוד B

```
 1. // strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.   static Groups groups;
 6.   vector<Groups::iterator> substrings;
 7.   static Groups::iterator joinGroup(const std::string& s) {
 8.     auto itr = groups.find(s);
 9.     if(itr == groups.end()) {
10.       // see explanation regarding map::insert at line 68
11.       itr = groups.insert({s, 1}).first;
12.     }
13.     else {
14.       ++(itr->second);
15.     }
16.     return itr;
17.   }
18.   static void leaveGroups(const vector<Groups::iterator>& subs) {
19.     for(auto itr: subs)
20.       if(--(itr->second) == 0) groups.erase(itr);
21.   }
22.   static void addOneToAll(const vector<Groups::iterator>& subs) {
23.     for(auto itr: subs) {
24.       ++(itr->second);
25.     }
26.   }
27. public:
28.   static void printGroups() {
29.     for(auto g : groups) {
30.       cout << '\"' << g.first << "\" (" << g.second << ")\n";
31.     }
32.   }
33.   MyString(const char* s = "") {
34.     substrings.push_back(joinGroup(s));
35.   }
36.   ~MyString() { leaveGroups(substrings); }
37.   MyString(const MyString& s): substrings(s.substrings) {
38.     addOneToAll(substrings);
39.   }
40.   MyString& operator=(const MyString& s) {
41.     if(this != &s) {
42.         leaveGroups(substrings);
43.         substrings = s.substrings;
44.         addOneToAll(substrings);
45.     }
46.     return *this;
47.   }
```

```
48.   MyString& operator+=(const MyString& s) {
49.     // below we insert all elements in s.substrings
50.     // into the end of this->substrings
51.     // (i.e. concatanating s.substrings into this->substrings)
52.     substrings.insert(/*insert before:*/ substrings.end(),
53.                       /*a copy of all elements in the range:*/
54.                       s.substrings.begin(), s.substrings.end());
55.     // now count me in for all the substrings that were added
56.     addOneToAll(s.substrings);
57.     return *this;
58.   }
59.   friend ostream& operator<<(ostream& out, const MyString& s) {
60.     for(auto itr: s.substrings) out << itr->first;
61.     return out;
62.   }
63. };
64.
65. // in MyString.cpp
66. Groups MyString::groups;
67.
68. // explanation regarding map::insert (line 11):
69. // map.insert returns std::pair<iterator, bool>
70. // 'first' is an iterator pointing to the new item or to
71. // an item with the same key that already existed, which
72. // prevented the insertion
73. // 'second' is whether insertion was performed
74.
75. int main() {
76.     MyString s1 = "Hello";
77.     MyString* s2 = new MyString(" World");
78.     MyString s3 = "Hello";
79.     s3 += *s2;
80.     cout << s3 << endl;
81.     MyString::printGroups();
82.     delete s2;
83.     // s1 = "!!!";
84.     // s3 += s1;
85.     // cout << s3 << endl;
86.     MyString::printGroups();
87. }
```

קטע קוד C – לשאלות הבונוס

```
 1. // strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.   static Groups groups;
 6.   list<Groups::iterator> substrings;
 7.   static Groups::iterator joinGroup(const std::string& s) {
 8.     auto itr = groups.find(s);
 9.     if(itr == groups.end()) {
10.       // explanation regarding map::insert appears in code B
11.       itr = groups.insert({s, 1}).first;
12.     }
13.     else {
14.       ++(itr->second);
15.     }
16.     return itr;
17.   }
18.   static void leaveGroups(const list<Groups::iterator>& subs) {
19.     for(auto itr: subs)
20.       if(--(itr->second) == 0) groups.erase(itr);
21.   }
22.   static void addOneToAll(const list<Groups::iterator>& subs) {
23.     for(auto itr: subs) {
24.       ++(itr->second);
25.     }
26.   }
27. public:
28.   static void printGroups() {
29.     for(auto g : groups)
30.       cout << '\"' << g.first << "\" (" << g.second << ")\n";
31.   }
32.   MyString(const char* s = "") {
33.     substrings.push_back(joinGroup(s));
34.   }
35.   ~MyString() { leaveGroups(substrings); }
36.   MyString(const MyString& s): substrings(s.substrings) {
37.     addOneToAll(substrings);
38.   }
39.   MyString(MyString&& s): substrings(std::move(s.substrings)) {}
40.   MyString& operator=(MyString s) {
41.     leaveGroups(substrings);
42.     substrings = std::move(s.substrings);
43.     return *this;
44.   }
45.   friend ostream& operator<<(ostream& out, const MyString& s) {
46.     for(auto itr: s.substrings) out << itr->first;
47.     return out;
48.   }
```

```
  MString& operator+=(MString&& s) {
    // the line below *moves* all elements in s.substrings
    // to the end of this->substrings
    substrings.splice(
      /*insert to my end:*/ substrings.end(),
      /*elements from:*/ s.substrings,
      /*in the range:*/ s.substrings.begin(), s.substrings.end());
    return *this;
  }
};

// in MyString.cpp
Groups MyString::groups;

int main() {
    MyString s1 = "Hello";
    MyString* s2 = new MyString(" World");
    MyString s3 = std::move(s1);
    // s3 += *s2;
    cout << s3 << endl;
    MyString::printGroups();
    delete s2;
    // s3 = s1 = "!!!";
    s3 += std::move(s1);
    cout << s3 << endl;
    MyString::printGroups();
}
```

בהצלחה !